

Contents

Chapter 29: Architecture and Product Review	1
1. The Architecture Review as an Engineering Artifact	2
2. The 30-Minute Structure	2
3. Section 1 — Problem	3
3.1 Quantify the Problem	4
4. Section 2 — Product	4
5. Product Scope	5
6. Section 3 — Architecture	6
7. Architecture Should Explain Decisions	6
8. Architecture Trade-offs	7
9. Section 4 — AI System	8
10. AI System Decomposition	8
11. Why AI Rather Than Conventional Software?	9
12. Section 5 — Evaluation	10
13. Baselines Matter	10
14. Section 6 — Economics	11
15. Economics Are Part of Architecture	11
16. Section 7 — Failures	12
17. Failure Is a Design Property	13
18. Section 8 — Roadmap	14
19. The Roadmap Should Follow the Bottleneck	15
20. Architecture Review Questions	15
21. The Presentation Should Tell One Story	17
22. The Architecture Review as a Design Audit	17
23. From Project to Engineering Judgment	18
24. Key Takeaways	19

Chapter 29: Architecture and Product Review

A strong AI engineer must be able to do more than build a working system.

They must be able to **explain why the system exists, why it was designed the way it was, what evidence supports its effectiveness, where it fails, and what should happen next.**

That is the purpose of the architecture and product review.

The exercise is deliberately constrained to a **30-minute presentation**. The constraint matters. A good architecture review is not a tour through every implementation detail. It is an argument.

The presenter should be able to establish:

Problem → Product → Architecture → AI → Evidence → Economics → Failures → Roadmap

The presentation should tell a coherent story from user problem to engineering solution.

1. The Architecture Review as an Engineering Artifact

An architecture review serves several purposes simultaneously.

It is:

- a product review
- a system-design review
- an AI design review
- an evaluation review
- an operational review
- an economic review
- a roadmap discussion

The central question is:

Did we build the right system, and did we build it correctly?

These are different questions.

Building the right system This is primarily a product question:

User Problem → Desired Outcome → Product

Building the system correctly This is primarily an engineering question:

Requirements → Architecture → Implementation → Validation

A project can succeed at one and fail at the other.

For example, an AI assistant might be technically excellent but solve a problem users do not care about.

Conversely, a product might solve an important problem but have an architecture that is too expensive, unreliable, insecure, or difficult to operate.

The review should expose both classes of failure.

2. The 30-Minute Structure

A useful allocation is:

Section	Time
Problem	3 min
Product	4 min
Architecture	6 min
AI system	4 min
Evaluation	4 min
Economics	3 min
Failures	3 min
Roadmap	3 min
Total	30 min

The architecture receives the largest allocation because the review should demonstrate engineering depth.

But the architecture should never dominate the presentation at the expense of the problem and product.

A common failure mode is spending 20 minutes explaining infrastructure and then saying:

“And users seem to like it.”

That reverses the priority.

The architecture exists to serve the product.

3. Section 1 — Problem

The presentation begins with the problem, not the technology.

Answer:

What are we solving?

A good problem statement identifies:

1. the user
2. the current workflow
3. the pain point
4. the desired outcome
5. why existing solutions are inadequate

For example:

Researchers spend hours searching across a large document collection to answer questions that require synthesizing information from multiple sources.

That is better than:

We built an agentic RAG system using a frontier model.
The second statement describes technology.
The first describes a problem.

3.1 Quantify the Problem

Whenever possible, quantify the baseline.
For example:

$$T_{\text{manual}} = 45 \text{ min}$$

versus:

$$T_{\text{system}} = 5 \text{ min}$$

or:

$$E_{\text{manual}} = 12\%$$

versus:

$$E_{\text{system}} = 5\%$$

The goal is to establish a measurable baseline against which the product can be evaluated.

The problem statement should therefore eventually become:

Current State → Pain → Desired State

4. Section 2 — Product

Now answer:

What did we build?

This section should demonstrate the actual user experience.

Show the product.

Do not spend four minutes describing what the product does when you can demonstrate it in 30 seconds.

A useful product description identifies:

- primary user
- primary workflow
- core capabilities
- user interface
- major interactions
- output
- value proposition

The product should be described in terms of **jobs to be done**, not implementation components.

For example:

“The user uploads a set of documents, asks a question, and receives an evidence-backed answer with citations.”

is better than:

“The application invokes an embedding model, queries a vector store, constructs a context window, and calls an LLM.”

The latter belongs in the architecture section.

5. Product Scope

Explicitly define what the product does **and does not do**.

A useful formulation is:

$$\text{Product Scope} = \{\text{Capabilities, Non-goals}\}$$

For example:

Does

- search documents
- answer questions
- cite evidence
- summarize findings
- maintain conversation state

Does not

- make autonomous high-stakes decisions
- modify source documents
- access arbitrary external systems

- guarantee correctness

This prevents architecture discussions from being disconnected from product requirements.

6. Section 3 — Architecture

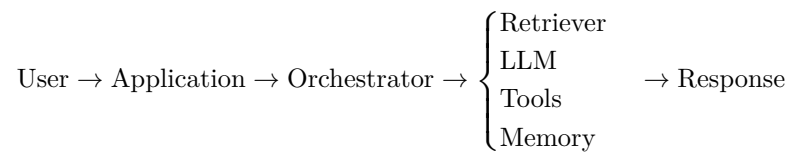
Now answer:

How does it work?

This is where the engineering depth becomes visible.

Start with the simplest possible architecture diagram.

For example:



Then expand the architecture only as necessary.

A good architecture diagram should make the major flows obvious.

7. Architecture Should Explain Decisions

Do not merely show components.

Explain **why they exist**.

For each major component, answer:

- What requirement does it satisfy?
- What alternative did we consider?
- Why did we choose this implementation?
- What trade-off does the decision introduce?

For example:

Retrieval Why retrieval?

Because the model's parametric knowledge is insufficient for the application's private or dynamic data.

Reranking Why reranking?

Because initial semantic retrieval provides high recall but insufficient precision for the generation context.

Structured output Why structured output?

Because downstream software requires machine-readable results rather than unconstrained natural language.

Tool calling Why tools?

Because the model needs access to external state or deterministic computation.

This transforms the architecture diagram from a collection of boxes into an engineering argument.

8. Architecture Trade-offs

Every meaningful architectural decision has trade-offs.

Examples:

Latency $\leftrightarrow$ Quality

Cost $\leftrightarrow$ Model Capability

Recall $\leftrightarrow$ Context Size

Autonomy $\leftrightarrow$ Control

Simplicity $\leftrightarrow$ Flexibility

A mature architecture review explicitly identifies these trade-offs.

For example:

We introduced a reranking stage because retrieval precision was insufficient, accepting approximately 150 ms of additional latency in exchange for a measurable improvement in grounded answer quality.

That is an engineering decision.

9. Section 4 — AI System

Now answer:

Why did we use AI here?

This question is more important than:

“Which model did we use?”

AI should be introduced because it provides a capability that conventional software cannot provide economically or effectively.

Potential reasons include:

- natural-language understanding
- unstructured information extraction
- semantic search
- synthesis
- classification
- generation
- reasoning over heterogeneous information
- adaptive tool selection
- multimodal interpretation

But not every problem requires an LLM.

If deterministic code can solve a problem more reliably, quickly, cheaply, and predictably, use deterministic code.

10. AI System Decomposition

Explain the AI system as a set of responsibilities.

For example:

AI System = Model, Context, Retrieval, Tools, Memory, Verification

For each component, explain its role.

Model What capability does the model provide?

Context What information is placed into the model’s context?

Retrieval How is relevant external knowledge selected?

Tools What actions or deterministic computations can the system perform?

Memory What information persists across interactions?

Verification How does the system determine whether the result is acceptable?

This decomposition makes it possible to distinguish **model intelligence** from **system intelligence**.

11. Why AI Rather Than Conventional Software?

A strong review should explicitly justify the AI boundary.

For every AI component, ask:

Why AI?

For every deterministic component, ask:

Why not AI?

This leads to an important architectural principle:

Use probabilistic components where flexibility is valuable and deterministic components where correctness is required.

For example:

LLM $\rightarrow$ interpret intent

followed by:

deterministic code $\rightarrow$ validate parameters

followed by:

tool $\rightarrow$ perform action

This hybrid architecture is often more reliable than allowing the model to control the entire system.

12. Section 5 — Evaluation

Now answer:

How do we know it works?

This section should present evidence, not assertions.

Show:

- evaluation dataset
- metrics
- baselines
- test results
- failure rates
- latency
- cost
- user-study results
- security findings

For AI systems, include both component-level and end-to-end evaluation.

For example:

Retrieval → Generation → Citation → Task Outcome

Evaluate each stage independently.

Then evaluate the complete workflow.

13. Baselines Matter

A result is difficult to interpret without a baseline.

Compare against:

- no-AI workflow
- simple prompting
- naive RAG
- smaller model
- previous architecture
- human performance
- existing product

For example:

System	Accuracy	Latency	Cost
Manual	82%	45 min	\$20
Baseline AI	87%	8 s	\$0.08

System	Accuracy	Latency	Cost
Final system	94%	4 s	\$0.05

Now the engineering improvements become meaningful.

14. Section 6 — Economics

Answer:

What does it cost?

AI systems introduce variable inference costs that traditional software often does not have.

Estimate:

$$C_{\text{request}} = C_{\text{LLM}} + C_{\text{embedding}} + C_{\text{retrieval}} + C_{\text{tools}} + C_{\text{infrastructure}}$$

Then estimate:

$$C_{\text{user/month}}$$

and at scale:

$$C_{\text{monthly}} = N_{\text{requests}} \times C_{\text{request}} + C_{\text{fixed}}$$

This allows the team to reason about the economics of deployment.

15. Economics Are Part of Architecture

Cost should not be treated as a finance issue that appears after architecture is complete.

Architecture determines economics.

For example:

More retrieval → more tokens → higher model cost

and:

More agent steps → more inference calls → higher latency and cost

Similarly:

larger model $\rightarrow$ higher quality $\rightarrow$ higher cost

The engineering objective is therefore not simply:

max quality

but something closer to:

$$\max \frac{\text{quality} \times \text{user value}}{\text{cost} \times \text{latency} \times \text{risk}}$$

This is why model selection, context engineering, retrieval, caching, routing, and workflow design are economic decisions as well as technical decisions.

16. Section 7 — Failures

This may be the most important section of the presentation.

Answer:

Where does it fail?

Do not hide failures.

A sophisticated engineering review makes them explicit.

Categorize failures into:

- model failures
- retrieval failures
- tool failures
- context failures
- infrastructure failures
- security failures
- usability failures
- product failures

For each major failure, explain:

1. what happens
2. why it happens
3. how often it happens
4. how severe it is
5. whether it is detected
6. whether it is recoverable

7. what mitigation exists

A useful representation is:

Failure $\rightarrow$ Cause $\rightarrow$ Impact $\rightarrow$ Mitigation

17. Failure Is a Design Property

A mature system is not defined by having no failures.

It is defined by having **controlled failure modes**.

For example:

Unknown answer $\rightarrow$ Abstain

is better than:

Unknown answer $\rightarrow$ Hallucinate

Similarly:

Tool unavailable $\rightarrow$ Retry $\rightarrow$ Fallback

is better than:

Tool unavailable $\rightarrow$ Agent crashes

The architecture should therefore specify not only the normal path:

Input $\rightarrow$ Processing $\rightarrow$ Output

but also:

Failure $\rightarrow$ Detection $\rightarrow$ Recovery $\rightarrow$ Safe degradation

18. Section 8 — Roadmap

The final question is:

What would we build next?

Do not turn the roadmap into a random feature wishlist.

Prioritize improvements according to expected value.

A useful framework is:

$$\text{Priority} \approx \frac{\text{Expected Impact} \times \text{Confidence}}{\text{Cost} \times \text{Risk}}$$

Potential roadmap categories include:

Product

- additional workflows
- better UX
- integrations
- personalization

AI

- better retrieval
- improved prompting
- model routing
- fine-tuning
- improved tool selection
- stronger verification

Infrastructure

- caching
- scaling
- lower latency
- observability
- reliability

Economics

- cheaper models
- fewer model calls
- smaller context
- intelligent routing

Security

- stronger isolation
- permission controls
- adversarial testing
- improved data governance

The roadmap should follow directly from the evidence presented earlier.

If evaluation shows retrieval is the dominant failure mode, the roadmap should prioritize retrieval.

If users rarely use a particular feature, building more features may be the wrong next step.

19. The Roadmap Should Follow the Bottleneck

One of the most useful engineering principles is:

Next Investment $\approx$ Largest Constraint on User Value

Suppose:

- model accuracy = 95%
- retrieval accuracy = 72%
- latency = 2 seconds
- cost = \$0.03/request

Improving the model from 95% to 96% may have little impact.

Improving retrieval from 72% to 90% may transform the product.

Similarly, if:

- technical performance is excellent
- user retention is poor

then the next investment should probably not be another optimization to the inference pipeline.

The bottleneck may be product-market fit.

20. Architecture Review Questions

During the presentation, reviewers should be able to ask questions such as:

Product

- Who is the primary user?
- What problem is being solved?
- How important is the problem?
- What is the measurable outcome?

Architecture

- Why was this architecture chosen?
- What alternatives were considered?
- Where are the system bottlenecks?
- What are the major dependencies?
- What happens when each dependency fails?

AI

- Why is AI necessary?
- Which components are probabilistic?
- Which components are deterministic?
- How is context constructed?
- How is model output verified?

Evaluation

- What is the baseline?
- What is the evaluation dataset?
- How representative is it?
- Where does the system fail?
- How do you know the evaluation itself is valid?

Economics

- What does one request cost?
- What happens at 10x or 100x traffic?
- What is the largest cost driver?
- What architectural decisions affect cost?

Product

- Do users actually use it?
- Do they come back?
- What value does it create?
- Why would they choose it over alternatives?

These questions should be anticipated rather than discovered for the first time during the review.

21. The Presentation Should Tell One Story

The strongest presentations have a clear causal structure:

We identified a problem.

↓

We built a product to solve it.

↓

We designed this architecture because of the product requirements.

↓

We introduced AI because it provides capabilities conventional software cannot provide economically.

↓

We evaluated the system against explicit criteria.

↓

The results show where it works and where it fails.

↓

The economics tell us whether it can operate sustainably.

↓

The evidence determines what we should build next.

That is much stronger than presenting eight disconnected sections.

22. The Architecture Review as a Design Audit

At the end of Chapter 29, the team should be able to step back from the implementation and evaluate the entire system.

The review should expose whether there is alignment between:

Problem ↔ Product ↔ Architecture ↔ AI ↔ Evaluation ↔ Economics

Misalignment is a powerful diagnostic signal.

For example:

Problem → Product mismatch The product does not actually address the user’s highest-value problem.

Product → Architecture mismatch The architecture is optimized for capabilities users do not need.

Architecture → AI mismatch AI is being used where deterministic software would be superior.

AI → Evaluation mismatch The team measures benchmark performance rather than actual task success.

Evaluation → Economics mismatch The system achieves excellent quality at an economically unsustainable cost.

Economics → Product mismatch The product creates insufficient value to justify its operating cost.

These mismatches are often more important than individual implementation bugs.

23. From Project to Engineering Judgment

The deeper purpose of Chapter 29 is not presentation skill.

It is **engineering judgment**.

Building the system teaches you how to construct it.

Evaluating it teaches you how to measure it.

The architecture review teaches you how to reason about it as a complete system.

That requires moving between several abstraction levels:

User → Product → System → Component → Model → Infrastructure

and then moving back upward:

Infrastructure → System → Product → User Value

Strong AI engineers can operate at both levels.

They can discuss token budgets and inference latency, but they can also explain why those details matter to the user and the business.

24. Key Takeaways

1. **An architecture review is an engineering argument, not a feature tour.**
2. **Start with the problem, not the technology.** The purpose of the architecture is to solve a user problem.
3. **Separate product requirements from implementation details.** Users care about outcomes; engineers must explain how those outcomes are produced.
4. **Architecture diagrams should communicate decisions.** Every major component should have a reason to exist.
5. **Explain trade-offs explicitly.** Quality, latency, cost, reliability, autonomy, and complexity are interconnected.
6. **Justify the AI boundary.** Use AI where probabilistic flexibility creates value and deterministic software where predictability matters.
7. **Evaluation must provide evidence.** Baselines, metrics, datasets, failure rates, and user outcomes are more important than demonstrations.
8. **Economics belong inside architecture.** Model selection, context size, retrieval, tool calls, and agent loops directly determine operating cost.
9. **Failures should be presented openly.** A mature system is not failure-free; it has controlled and observable failure modes.
10. **The roadmap should follow evidence.** The next investment should target the largest constraint on user value rather than the most interesting technical feature.
11. **The eight sections should form one causal story:**

Problem → Product → Architecture → AI → Evidence → Economics → Failures → Next Step

12. **The ultimate skill being tested is engineering judgment.** You should be able to explain not only what you built, but why you built it, whether it works, what it costs, where it fails, and what should happen next.

The project is no longer merely an implementation.

It is now an **engineered system with an explicit technical, product, and economic thesis.**

The architecture review is where you defend that thesis.